ENSTA | INSTITUT POLYTECHNIQUE DE PARIS

Module 3 : Design Patterns GoF (Patrons de conception)

Création · Structure · Comportement

Python & Java

Création | Structure | Comportement

MODULE 3 · Ingénierie Logicielle Évaluation : 40% TP · 60% Contrôle Continu

La Roadmap du Cursus : Ingénierie Logicielle

01
Modélisation Objet Avancée

Classes, Héritage, Encapsulation, Polymorphisme

02
Principe SOLID & Qualité

5 Principes SOLID, Clean Code, Refactoring

03
Patrons de Conception

Singleton – Adapter – Strategy · Observer · GoF

04
Modélisation UML

Représentation visuelle des architectures

Objectif final : Concevoir · Implémenter · Tester · Livrer un logiciel de qualité industrielle

Fil Rouge : CompteBancaire traverse les 3 modules

Module 1
POO

CompteBancaire
+ CompteEpargne
+ CompteCheque
Héritage / Encapsulation/Polymorphisme

→

Module 2
SOLID

Refactoring SOLID
SRP → 4 classes
OCP → Ifrais
LSP , IRP
DIP → IRepo injecté

→

Module 3
PATTERNS

Singleton : config
Adapter : Conversion
Strategy : frais
Observer : alertes
GoF sur code SOLID

Architecture finale de CompteBancaire après les 3 modules :

CompteBancaire
(SRP / LSP)

retirer() → bool
__solde privé
IRepo injecté

IFrais Strategy
(OCP Pattern)

FraisCheque
FraisEpargne
FraisPro + N+1...

IObserver Observer
(Nouveau M3)

AlerteSMS
AlerteEmail
AlertePush + N+1...

IRepo DIP
(M2 reconduit)

RepoBDD
RepoMock (tests)
RepoCSV...

ConfigBanque
Singleton M3

instance unique
taux_interet
plafond_retrait

Les Design Patterns GoF : des solutions réutilisables à des problèmes récurrents

Définition

Un Design Pattern est une solution générique et réutilisable à un problème de conception récurrente dans votre code.

- Pas du code copié-collé
  Un schéma d'architecture
- Inspiré du BTP
  Architecture de l'Urbanisme (Organisation , Structuration , Normes)
- Langage commun entre développeurs
  «utilise un Strategy ici» est compris universellement
- Catalogue GoF : 23 patterns
  3 familles : Création, Structure, Comportement
- Avantages : Modélisation , qualité & standardisation bonne pratique, langage commun à tous , flexibilité et réutilisation du code

1- Création : instanciation , Configuration des classes et des objets Singleton Factory Method Builder	Comment créer des objets de façon flexible et sans couplage fort.
2- Structure : Organisation des classes dans une structure Adapter Decorator Façade	Comment assembler des objets en structures plus grandes et plus flexibles.
3- Comportement : Organisation des Objets en vue de leurs collaborations Strategy ✓ Observer ✓ Template Method	Comment les objets communiquent et se répartissent les tâches.

Ce module : Singleton , Decorator , Strategy + Observer sur CompteBancaire en cours · Lien explicite avec SOLID M2

Patrons de Création : Singleton , Factory method , Abstract Factory , Prototype , Builder

"Comment créer des objets de façon flexible, sans couplage fort avec leurs classes concrètes"

Pattern 1 Singleton

Instance unique garantie

Garantir qu'une classe n'a qu'une seule instance. Utilisé pour ConfigBanque, logs, connexion BDD.

Pattern 2 Factory Method

Créer sans couplage fort

Déléguer la création d'objets à des sous-classes. Le code client ne connaît pas la classe concrète.

Fil rouge CompteBancaire

ConfigBanque · CompteFactory

Singleton → ConfigBanque partagée. Factory → créer CompteCheque ou CompteEpargne sans new direct.

Pattern : Singleton — Instance Unique Globale

"Garantir qu'une classe n'a qu'une seule instance et fournir un point d'accès global"

Problème Sans Pattern

ConfigBanque créée 10 fois.

ConfigBanque créée 10 fois. Chaque instance a son propre taux. Incohérence totale.

Pattern Singleton

Une seule instance de Config

Une seule instance de ConfigBanque. Toutes les classes partagent le même taux.

Attention Limites

Le Singleton est souvent abu

Le Singleton est souvent abusé. Ne l'utiliser que pour ressource unique : config, logs.

Singleton : ConfigBanque instance unique — taux et plafond partagés

Cas d'usage réel : configuration de la banque (taux, plafond) doit être identique et fixe dans toutes les classes.

Python · Singleton ConfigBanque BON

```python
# Pattern Singleton — Python
class ConfigBanque:
  _instance = None  # attribut de classe

  def __new__(cls):  # méthode
    # Créer seulement si inexistant ( constructeur )
    if cls._instance is None:
      cls._instance = super().__new__(cls)     # création objet
      cls._instance.taux_interet = 0.035       # règle le taux
      cls._instance.plafond = 10000.0          # règle le plafond
    return cls._instance                   #retourne l'objet déjà crée

# Utilisation
config1 = ConfigBanque()     # même objet
config2 = ConfigBanque()
print(config1 is config2)  # True !
print(config1.taux_interet)  # 0.035

# Dans CompteBancaire
class CompteBancaire:
  def __init__(self, t, s, strat):
    self.config = ConfigBanque()  # singleton
    self.taux = self.config.taux_interet

# Singleton garantit que tous les comptes ont un taux et plafond fixe
```

Java · Singleton ConfigBanque (BON) BON

```java
// Pattern Singleton — Java
public class ConfigBanque {
  // 1. Instance privée statique
  private static ConfigBanque instance;       #verrouillage
  private double tauxInteret = 0.035;
  private double plafond = 10000.0;

  // 2. Constructeur privé
  private ConfigBanque() {}

  // 3. Méthode d'accès statique
  public static ConfigBanque getInstance() {
    if (instance == null) {
      instance = new ConfigBanque();
    }
    return instance;
  }
  public double getTauxInteret() {
    return tauxInteret; }
}
// Utilisation dans CompteBancaire
double taux =
  ConfigBanque.getInstance()
              .getTauxInteret();
```

Pattern : Factory Method (Fabrique)

Définit une interface pour la création d'Objet dans une classe mère , mais délègue des sous classes le choix d'objet à créer

Problème **Sans Pattern**

new direct = couplage fort

new direct = couplage forttype de frais force la modification du code existant. OCP violé.

Pattern **Factory Method**

Une méthode fabrique

Une méthode fabriqueest encapsulé dans une classe. Interchangeable au runtime.

Bénéfice **Résultat**

Ajouter CompteJeune

Ajouter CompteJeuner CompteBancaire. Code ouvert OCP, testable, extensible.

Factory Method · Python & Java · CompteFactory

CompteFactory crée des ICompte selon le type. Le client ne connaît jamais la classe concrète.
Factory : Délégué pour mieux régner

Python · ICompte + CompteFactory

BON

```python
# Pattern Factory Method — Python
# 1. Interface produit
from abc import ABC, abstractmethod
class ICompte(ABC):                                  #Interface : classe fantome qui définit les régles
  @abstractmethod
  def retirer(self,m:float)->bool: ...
  @abstractmethod
  def get_solde(self)->float: ...

# 2. Produits concrets                                      # sous classes rélles ( ouvriers)
class CompteCheque(ICompte):
  def __init__(self,t,s):
    self.__solde = s
  def retirer(self,m):
    if m<=self.__solde:
      self.__solde-=m; return True
    return False
  def get_solde(self): return self.__solde

class CompteEpargne(ICompte):
  def __init__(self,t,s,taux=0.035):
    self.__solde=s; self.__taux=taux
  def retirer(self,m):
    if m<=self.__solde:
      self.__solde-=m; return True
    return False
  def get_solde(self): return self.__solde

# 3. Factory — cœur du pattern
class CompteFactory:                                          # cerveau du pattern
  @staticmethod
  def creer(type_:str,t:str,s:float)->ICompte:                # methodes
    if type_ == 'cheque':
      return CompteCheque(t, s)
    elif type_ == 'epargne':
      return CompteEpargne(t, s)
    # CompteJeune : 1 ligne, 0 modif client        # decode l'objet à fabriquer chéque ou epargne
    raise ValueError(type_)

# 4. Client — ignore la classe concrète
c: ICompte = CompteFactory.creer(
  'epargne', 'Ali', 1000.0)
c.retirer(200)
print(c.get_solde())  # 800.0

# la variable c reçoit un compte epargne sans que tu l'écris c'est le factory qui fait le travail
 pour toi tu ulisr ensuite c.retirer ( 200)
```

Java · interface ICompte + CompteFactory

BON

```java
// Pattern Factory Method — Java
// 1. Interface produit
public interface ICompte {
  boolean retirer(double montant);
  double getSolde();
}

// 2. Produits concrets
public class CompteCheque implements ICompte {
  private double solde;
  public CompteCheque(String t,double s)
    { this.solde=s; }
  public boolean retirer(double m) {
    if(m<=solde){solde-=m;return true;}
    return false; }
  public double getSolde(){return solde;}
}

// 3. Factory — la clé du pattern
public class CompteFactory {
  public static ICompte creer(
      String type,String t,double s) {
    return switch(type) {
      case "cheque" ->
        new CompteCheque(t,s);
      case "epargne" ->
        new CompteEpargne(t,s);
      // CompteJeune: 1 ligne, 0 modif
      default -> throw new
        IllegalArgumentException(type);
    };
  }
}

// 4. Client — ignore la classe concrète
ICompte c = CompteFactory.creer(
  "epargne","Ali",1000.0);
c.retirer(200);
System.out.println(c.getSolde()); // 800
```


Ajouter CompteJeune = 1 classe. Zéro modif client. OCP respecté.

Patrons de Structure : Adapter , Decorator , Bridge , Composite , façade , Flyweight, Proxy

"Comment assembler des objets et des classes en structures plus grandes tout en restant flexibles"

Pattern 1 **Adapter**

Interfaces incompatibles

Interfaces incompatiblestype de frais force la modification du code existant. OCP violé.

Pattern 2 **Decorator**

Enrichir sans héritage

Enrichir sans héritageest encapsulé dans une classe. Interchangeable au runtime.

Fil rouge **CompteBancaire**

Adapter + Decorator

Adapter + Decoratorr CompteBancaire. Code ouvert OCP, testable, extensible.

Pattern : Adapter — Pont entre Interfaces Incompatibles

"Convertir l'interface d'une classe en une autre interface attendue par le client"

Problème **Sans Pattern**

Interface incompatible

Interface incompatibletype de frais force la modification du code existant. OCP violé.

Pattern **Adapter**

Classe Adapter intermédiaire

Classe Adapter intermédiaireest encapsulé dans une classe. Interchangeable au runtime.

Bénéfice **Résultat**

Intégration sans modification

Intégration sans modificationr CompteBancaire. Code ouvert OCP, testable, extensible.

Adapter · Python & Java · PaiementAdapter

PaiementAdapter implémente IRepo et wraps ServiceLegacy. CompteBancaire et ServiceLegacy restent intacts.

Python · PaiementAdapter wraps ServiceLegacy

BON

```python
# Pattern Adapter — Python
# 1. Interface cible (client l'utilise pour faire virement)
from abc import ABC, abstractmethod
class IRepo(ABC):                                          #montant
  @abstractmethod
  def virer(self,montant:float,dest:str)->bool: ...

# 2. Service legacy — interface incompatible
# (on ne le modifie JAMAIS) : vieux systéme
class ServicePaiementLegacy:                                #classe
  def effectuer_virement(self,                              #methode
      montant_centimes:int,code_dest:str)->int:
    print(f'Legacy: {montant_centimes}c')
    return 0  # 0=OK

# 3. Adapter — pont magique  entre les deux ( vieux et nouveau)
class PaiementAdapter(IRepo):             #classe qui traduit les ordres
  def __init__(self):                           # constructeur
    self._leg = ServicePaiementLegacy()
  def virer(self,montant:float,dest:str)->bool:             #méthode
    # Conversion float€ → centimes int
    centimes = int(montant * 100)                          # conversion
    code = dest.upper()[:8]                                # formatage
    return self._leg.effectuer_virement(
      centimes, code) == 0                        # appelle vieux service

# 4. Client — utilise IRepo, ignore Legacy ( Compte utilize Irepos)
repo: IRepo = PaiementAdapter()
ok = repo.virer(500.0, 'FR76...')
print(ok)  # True

# CompteBancaire utilise IRepo inchangée
class CompteBancaire:
  def __init__(self,t,s,repo:IRepo):
    self.__repo = repo
  def virer(self,m,dest):
    return self.__repo.virer(m, dest)
```

BON

```java
// Pattern Adapter — Java
// 1. Interface cible
public interface IRepo {
  boolean virer(double montant, String dest);
}

// 2. Service legacy incompatible
public class ServicePaiementLegacy {
  // Ne pas modifier ce service externe
  public int effectuerVirement(
      int montantCentimes, String codeDest) {
    System.out.println(
      "Legacy: "+montantCentimes+"c");
    return 0; // 0 = succès
  }
}

// 3. Adapter — implémente IRepo, wraps Legacy
public class PaiementAdapter implements IRepo {
  private ServicePaiementLegacy legacy
    = new ServicePaiementLegacy();

  public boolean virer(
      double montant, String dest) {
    // Conversion float€ → centimes
    int c = (int)(montant * 100);
    String code = dest
      .substring(0,Math.min(8,dest.length()))
      .toUpperCase();
    return legacy.effectuerVirement(c,code)==0;
  }
}

// 4. Client — ignore Legacy totalement
IRepo repo = new PaiementAdapter();
boolean ok = repo.virer(500.0,"FR76...");
// CompteBancaire inchangé | Legacy inchangé
```


ServiceLegacy intact. CompteBancaire intact. Adapter = seule couche modifiée. Zéro régression.

Pattern : Decorator — Enrichissement Dynamique

"Attacher dynamiquement des fonctionnalités supplémentaires à un objet, alternative flexible à l'héritage"

Problème **Sans Pattern**

Héritage explosif

Héritage explosiftype de frais force la modification du code existant. OCP violé.

Pattern **Decorator**

Wrapper en couches

Wrapper en couchesest encapsulé dans une classe. Interchangeable au runtime.

Bénéfice **Résultat**

Composition à la carte

Composition à la carter CompteBancaire. Code ouvert OCP, testable, extensible.

Decorator · Python & Java · LoggingDecorator

Composition LoggingDecorator(PlafondDecorator(compte)). Chaque couche testable isolément.

Python · LoggingDecorator + PlafondDecorator

BON

```python
# Pattern Decorator — Python
# 1. Interface commune
class ICompte(ABC):
  @abstractmethod
  def retirer(self,m:float)->bool: ...
  @abstractmethod
  def get_solde(self)->float: ...

# 2. Composant concret (à décorer)
class CompteBancaire(ICompte):                          # Classe de base ( fixe)
  def __init__(self,t,s): self.__solde=s
  def retirer(self,m):
    if m<=self.__solde:
      self.__solde-=m; return True
    return False
  def get_solde(self): return self.__solde

# 3. Decorator de base (délégation)                    # Décorateur base
class CompteDecorator(ICompte):
  def __init__(self,compte:ICompte):
    self._c = compte
  def retirer(self,m): return self._c.retirer(m)
  def get_solde(self): return self._c.get_solde()

# 4. Decorator : Logging                        # ecrit un message avant de retier
class LoggingDecorator(CompteDecorator):
  def retirer(self,m):
    print(f'[LOG] Retrait {m}...')
    ok = super().retirer(m)
    print(f'[LOG] OK={ok}')
    return ok

# 5. Decorator : Plafond        c  # avant d'autoriser le retrait verifie le plafond
class PlafondDecorator(CompteDecorator):
  def __init__(self,c,plafond=500.0):
    super().__init__(c); self._max=plafond
  def retirer(self,m):
    if m>self._max:
      print(f'Plafond {self._max}!'); return False
    return super().retirer(m)

# 6. Composition à la carte (runtime)     # Composition carte final ( log , plafond)
base   = CompteBancaire('Ali', 2000)
secure = LoggingDecorator(
           PlafondDecorator(base, 500))
secure.retirer(300)  # loggé + OK
secure.retirer(600)  # bloqué plafond
```

Java · LoggingDecorator + PlafondDecorator

BON

Activer/désactiver logging au runtime.
Chaque couche testable. OCP respecté

```java
// Pattern Decorator — Java
// 1. Interface commune
public interface ICompte {
  boolean retirer(double montant);
  double getSolde();
}

// 2. Composant concret
public class CompteBancaire implements ICompte {
  private double solde;
  public CompteBancaire(String t,double s){solde=s;}
  public boolean retirer(double m){
    if(m<=solde){solde-=m;return true;}return false;}
  public double getSolde(){return solde;}
}

// 3. Decorator abstrait (délégation)
public abstract class CompteDecorator
    implements ICompte {
  protected ICompte c;
  public CompteDecorator(ICompte c){this.c=c;}
  public boolean retirer(double m){return c.retirer(m);}
  public double getSolde(){return c.getSolde();}
}

// 4. Decorator : Logging
public class LoggingDecorator extends CompteDecorator {
  public LoggingDecorator(ICompte c){super(c);}
  @Override
  public boolean retirer(double m){
    System.out.println("[LOG] Retrait "+m);
    boolean ok=super.retirer(m);
    System.out.println("[LOG] OK="+ok);
    return ok;}
}

// 5. Decorator : Plafond
public class PlafondDecorator extends CompteDecorator {
  private double max;
  public PlafondDecorator(ICompte c,double p)
    {super(c); max=p;}
  @Override
  public boolean retirer(double m){
    if(m>max){System.out.println("Plafond!");
      return false;}
    return super.retirer(m);}
}

// 6. Composition à la carte
ICompte base = new CompteBancaire("Ali",2000.0);
ICompte s = new LoggingDecorator(
  new PlafondDecorator(base,500.0));
s.retirer(300); // loggé + OK
s.retirer(600); // bloqué par plafond
```

Patrons de Comportement : Strategy, Observer , Command , Iterator , Mediator

"Comment les objets communiquent, se coordonnent et se répartissent les responsabilités"

Pattern 1 **Strategy**

Algorithme interchangeable

Algorithme interchangeabletype de frais force la modification du code existant. OCP violé.

Pattern 2 **Observer**

Notification automatique

Notification automatiqueest encapsulé dans une classe. Interchangeable au runtime.

Fil rouge **CompteBancaire**

Strategy + Observer sur CompteBancaire

Strategy + Observer sur CompteBancairer CompteBancaire. Code ouvert OCP, testable, extensible.

Pattern : Strategy — Comportement Interchangeable

"Définir une famille d'algorithmes, les encapsuler et les rendre interchangeables«
Exemple : Passer des frais chèques aux frais épargnes pendant que le programme tourne

Problème **Sans Pattern**

if/elif infini : ajouter un

if/elif infini : ajouter un type de frais force la modification du code existant. OCP violé.

Pattern **Strategy**

Chaque algorithme de calcul

Chaque algorithme de calcul est encapsulé dans une classe. Interchangeable au runtime.

Bénéfice **Résultat**

Ajouter FraisPro sans touche

Ajouter FraisPro sans toucher CompteBancaire. Code ouvert OCP, testable, extensible.

Sans Strategy : le if/elif interminable — OCP et SRP violés

Problème : tout ajout de type (FraisPro, FraisJeune, FraisEntreprise) force à modifier CompteBancaire. Fragile.

Python · if/elif (SANS Strategy) MAUVAIS

```python
class CompteBancaire:
  def calculer_frais(self, type_compte):
    # SANS Strategy : if/elif infini
    # Chaque nouveau type = modifier ici
    # OCP violé, SRP violé
    if type_compte == 'cheque':
      return self.__solde * 0.02
    elif type_compte == 'epargne':
      return self.__solde * 0.005
    elif type_compte == 'premium':
      return 0.0
    # Ajouter 'pro' ou 'jeune' ?
    # → modifier CETTE classe
    # → risque de casser cheque/epargne
```

Java · switch (SANS Strategy) MAUVAIS

```java
public class CompteBancaire {
  double calculerFrais(String type) {
    // SANS Strategy : switch infini
    switch (type) {
      case "cheque":
        return solde * 0.02;
      case "epargne":
        return solde * 0.005;
      case "premium":
        return 0.0;
      // Ajouter "pro" ?
      // Modifier ICI = OCP violé
      default: return 0;
    }
  }
}
```

Signal : if/elif ou switch sur un type d'objet → Pattern Strategy à appliquer. Lien direct avec OCP du Module 2.

Pattern Strategy : Python vs Java — Différences Clés

Élément	Python	Java
Interface Strategy	class IFrais: (ABC ou classe parent)	interface IFrais { double calculer(); }
Implémentation	class FraisCheque(IFrais):	class FraisCheque implements IFrais {
Injection (DIP)	def __init__(self, ..., s: IFrais):	public CB(String t, double s, IFrais st)
Swap runtime	compte.set_strategie(FraisEpargne())	compte.setStrategie(new FraisEpargne())
Vérification	Duck typing — runtime Python	Compile-time — erreur si calculer() manque
Ajout type	Nouvelle classe, 0 modif existante	Nouvelle classe, 0 modif existante

Pattern : Observer — Notification Automatique

"Un objet notifie automatiquement tous ses abonnés lors de chaque changement d'état«
Exemple : Alertes (message , emails , notification)

Problème Sans Pattern

CompteBancaire appelle direc

CompteBancaire appelle directement SMS, Email, Log. Couplage fort, DIP violé.

Pattern Observer

CompteBancaire publie un évé

CompteBancaire publie un événement. Les abonnés reçoivent automatiquement la notification.

Bénéfi ce Résultat

Ajouter AlertePush sans touc

Ajouter AlertePush sans toucher CompteBancaire. Découplage total Publisher/Subscriber.

Sans Observer : appels directs dans retirer() — DIP + SRP + OCP violés

Problème : CompteBancaire connaît SMS, Email, Log. Ajouter AlertePush = modifier retirer(). Code fragile et non extensible.

Python · Appels directs (SANS Observer) MAUVAIS

```python
class CompteBancaire:
  def retirer(self, montant):
    if montant <= self.__solde:
      self.__solde -= montant
      # SANS Observer : appels directs
      # DIP violé, SRP violé, OCP violé
      self.sms.envoyer(
        f'Retrait {montant}€')
      self.email.notifier(
        f'Solde: {self.__solde}€')
      self.log.ecrire(
        f'Transaction: -{montant}')
      # Ajouter AlertePush ?
      # → modifier retirer() !
      return True
    return False
```

Java · Couplage fort (SANS Observer) MAUVAIS

```java
public class CompteBancaire {
  private ServiceSMS sms;
  private ServiceEmail email;
  private Logger log;

  public boolean retirer(double montant) {
    if (montant <= solde) {
      solde -= montant;
      // SANS Observer : couplage fort
      sms.envoyer("Retrait "+montant);
      email.notifier("Solde: "+solde);
      log.ecrire("Transaction");
      // AlertePush ? modifier ici !
      return true;
    }
    return false;
  }
}
```

Signal : une méthode métier qui appelle plusieurs services externes → Pattern Observer à appliquer.

Strategy + Observer ensemble sur CompteBancaire — main() Python & Java

Démonstration : 1 compte, 1 Strategy, 3 Observers abonnés. Retrait déclenche notifications. Swap de stratégie sans arrêt.

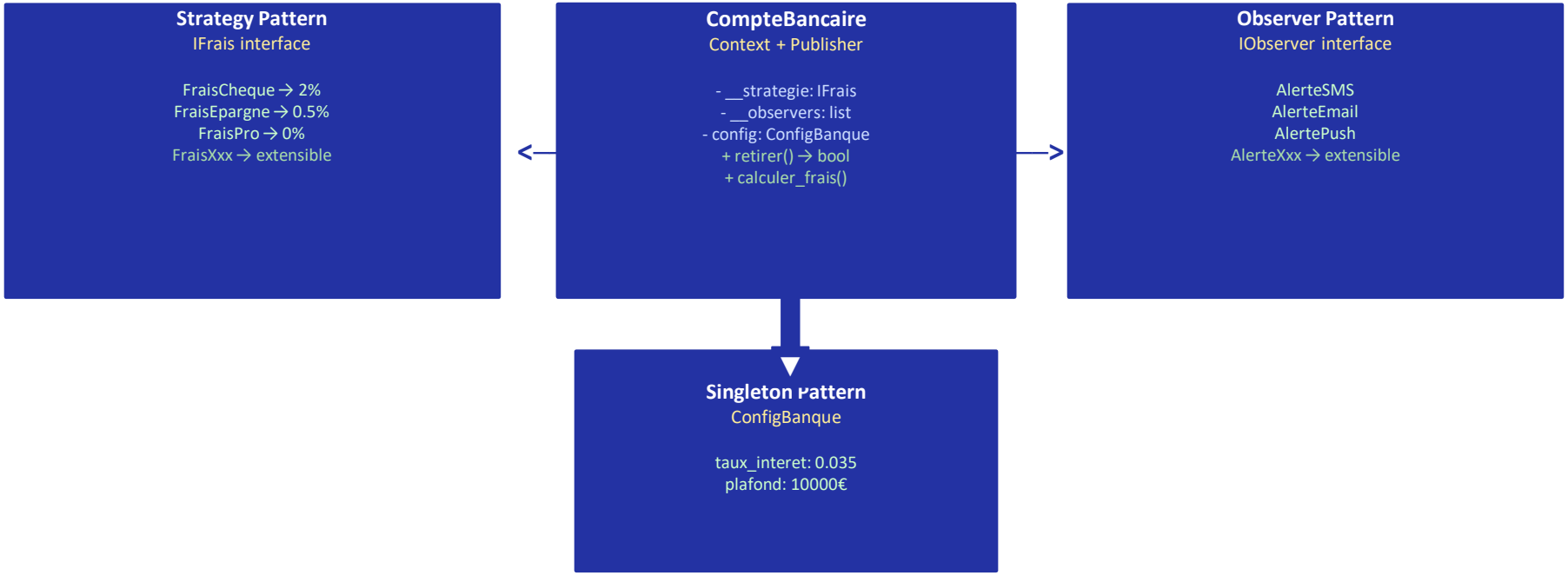
Python · main() Strategy + Observer complet (BON) BON

```python
# Utilisation complète : Strategy + Observer
# sur CompteBancaire — main() Python

# Création avec Strategy FraisCheque
compte = CompteBancaire(
  'Julien Moreau', 2000.0, FraisCheque())

# Abonnement de 3 observers
compte.abonner(AlerteSMS())
compte.abonner(AlerteEmail())
compte.abonner(AlertePush())

# Retrait : déclenche Observer auto
compte.retirer(500)
# → SMS:   Retrait 500€ effectué
# → Email: Solde restant 1500€
# → Push:  retrait reçu

# Calcul frais via Strategy
frais = compte.calculer_frais()
print(f'Frais: {frais}€')  # 30.0€

# Changer de stratégie au runtime
compte.set_strategie(FraisEpargne())
frais2 = compte.calculer_frais()
print(f'Frais épargne: {frais2}€')  # 7.5€
```

Java · main() Strategy + Observer complet (BON) BON

```java
// Utilisation complète : Strategy + Observer
// CompteBancaire — main() Java

// Création avec Strategy FraisCheque
CompteBancaire compte = new CompteBancaire(
  "Julien Moreau", 2000.0,
  new FraisCheque());

// Abonnement de 3 observers
compte.abonner(new AlerteSMS());
compte.abonner(new AlerteEmail());
compte.abonner(new AlertePush());

// Retrait : déclenche Observer auto
compte.retirer(500);
// → SMS:   Retrait 500€ effectué
// → Email: Solde restant 1500.0€
// → Push:  retrait reçu

// Calcul frais via Strategy
double frais = compte.calculerFrais();
System.out.println(frais); // 30.0

// Changer de stratégie au runtime
compte.setStrategie(new FraisEpargne());
System.out.println(
  compte.calculerFrais()); // 7.5
```

Architecture Complète : CompteBancaire avec les 3 Patterns GoF



Flux retirer(500) : CompteBancaire → applique IFrais.calculer() → soustrait 500 → notifie tous IObserver → lit ConfigBanque.plafond

SOLID et Design Patterns : une relation directe et intentionnelle

Principe SOLID	Problème résolu	Pattern appliqué	Sur CompteBancaire
OCP	if/elif frais infini	Strategy	IFrais → FraisCheque, FraisEpargne...
OCP + DIP	Appels directs SMS/Email	Observer	IObserver → AlerteSMS, AlerteEmail...
DIP	new ConcreteClass() dur	Strategy + Observer	IFrais et IObserver injectés
SRP	CompteBancaire fait tout	S + O + Singleton	Chaque pattern isole une responsabilité
ISP	Interface trop grosse	Interfaces séparées	IFrais ≠ IObserver ≠ IRepo

Création· Structure · Comportement : Python vs Java — Résumé

Pattern	Rôle	Python	Java
Singleton	Garantir une **instance unique** pour un service global ex: ConfigBanque	class CompteBanque: / def calculer()	interface CompteBanque { double calculer(); }
Strategy	Injection dans Context	def __init__(..., strat: IFrais)	public CB(IFrais strat)
Observer	Abonnement	compte.abonner(AlerteSMS())	compte.abonner(new AlerteSMS())
Observer	Notification	for obs in __observers: obs.update(e)	for (IObserver o : observers) o.update(e);
Observer	Ajout subscriber	Nouvelle classe IObserver, 0 modif	Nouvelle classe implements IObserver
Singleton	Instance unique	if cls._instance is None: créer	if (instance == null) instance = new CB()

Mindset Ingénieur : 3 Réflexes pour Choisir le Bon Pattern

S

Je vois un if/elif ou switch sur un type d'objet dans une méthode métier ?
→ Pattern STRATEGY · Remplacer par interface + classes concrètes · Lien : OCP M2

O

Une méthode métier appelle directement plusieurs services (SMS, Email, Log) ?
→ Pattern OBSERVER · Découpler via IObserver + liste d'abonnés · Lien : DIP + OCP M2

SG

Un objet de configuration est créé plusieurs fois dans différentes classes ?
→ Pattern SINGLETON · Instance unique ConfigBanque · Prudence : difficile à tester.

Règle d'or des Design Patterns

Ne pas appliquer un pattern par défaut. Identifier d'abord le problème :

- Algorithme variable → Strategy
- N abonnés à notifier → Observer
- Ressource unique → Singleton (avec prudence)

Sur-utiliser les patterns = over-engineering. Sous-utiliser = code spaghetti.

Réflexe : lis le code de CompteBancaire et demande-toi quelles interfaces manquent pour le rendre extensible.

Fin du Module 3

Place à la Pratique DP !

Travaux Pratiques (TP) : Python & Java : **Identifiez les patterns adaptés sur Gestion des étudiants TP 1 / HEP**

Objectif : Strategy + Observer + Singleton+Adapter sur Gestion des Etudiants · 2h

01	02	03	04
Création	**Structure**	**Comportement**	**Main()** Demo complète